Object Oriented Programming

A Tale of Constructors

IT560

Sourish Dasgupta
Assistant Professor, DA-IICT, Gandhinagar
https://daiict.ac.in/faculty/sourish-dasgupta

```java
public class Person {
    private String name;
    private int age;

    // Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }

    // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Method to display person details
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
```

```java
    public static void main(String[] args) {
        // Using the default constructor
        Person person1 = new Person();
        person1.display();

        // Using the parameterized constructor
        Person person2 = new Person("Alice", 25);
        person2.display();
    }
}
```

Guess the language?

```cpp
class Person {
private:
    string name;
    int age;

public:
    // Default constructor
    Person() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized constructor
    Person(string personName, int personAge) {
        name = personName;
        age = personAge;
    }

    // Method to display person details
    void display() const {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    // Using the default constructor
    Person person1;
    person1.display();

    // Using the parameterized constructor
    Person person2("Alice", 25);
    person2.display();

    return 0;
}
```

Guess the language?

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
person = Person("Alice", 25)  # Constructor initializes name and age
```

Guess the language?

Why Constructors? OOP begins at home!

Purpose	Description
Object Initialization	Sets up the *initial state* of an object by initializing its attributes
Encapsulation	Hides *initialization details*, ensuring proper setup and reducing errors
Abstraction	Simplifies object creation by abstracting the *setup logic*
Code Reusability	Enables consistent reuse of *initialization logic* for multiple objects

Why Constructors? OOP begins at home!

Purpose	Description
Automatic Invocation	Ensures the *constructor is called automatically* when an object is created
Custom Behavior	Allows *defining specific behaviors* (e.g., validations, derived properties) during object creation
Overloading	(In some languages) Provides flexibility by allowing *multiple ways to initialize* an object
Improved Readability	Enhances code clarity and usability by *standardizing* object creation and initialization

Are Constructors good enough?

What’s wrong with this?

```python
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

# Client Code
def animal_sound(animal_type):
    if animal_type == "dog":
        animal = Dog()  # Tight coupling to the Dog class
    elif animal_type == "cat":
        animal = Cat()  # Tight coupling to the Cat class
    else:
        raise ValueError("Unknown animal type")

    return animal.speak()

# Usage
print(animal_sound("dog"))  # Output: Woof!
print(animal_sound("cat"))  # Output: Me
```

Explicit Constructor is not needed if no setup needed

```python
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

# Client Code
def animal_sound(animal_type):
    if animal_type == "dog":
        animal = Dog()  # Tight coupling to the Dog class
    elif animal_type == "cat":
        animal = Cat()  # Tight coupling to the Cat class
    else:
        raise ValueError("Unknown animal type")

    return animal.speak()

# Usage
print(animal_sound("dog"))  # Output: Woof!
print(animal_sound("cat"))  # Output: Me
```

```python
class Animal:
    def __init__(self):
        print("Animal constructor called")

class Dog(Animal):
    def __init__(self):
        super().__init__()  # Call the Animal constructor
        print("Dog constructor called")

class Cat(Animal):
    def __init__(self):
        super().__init__()  # Call the Animal constructor
        print("Cat constructor called")
```

Optional!

But there's something genuinely problematic!

```python
# Client Code
def animal_sound(animal_type):
    if animal_type == "dog":
        animal = Dog()  # Tight coupling to the Dog class
    elif animal_type == "cat":
        animal = Cat()  # Tight coupling to the Cat class
    else:
        raise ValueError("Unknown animal type")

    return animal.speak()
```

Adding a *new* animal *type* requires modifying `animal_sound`, violating the **Open-Closed Principle** (OCP)

The function is limited to the *specific* classes (Dog and Cat) and cannot handle new types *dynamically*

Mocking or *replacing* Dog or Cat in tests is difficult because they are hardcoded

Tight Coupling!

Decoupling Constructors

```python
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"
```

```python
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"
```

Decoupling Constructors

```python
# Client Code
def animal_sound(animal_type):
    if animal_type == "dog":
        animal = Dog()  # Tight coupling t
    elif animal_type == "cat":
        animal = Cat()  # Tight coupling t
    else:
        raise ValueError("Unknown animal t

    return animal.speak()
```

```python
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Unknown animal type")
```

Adding a new animal type (e.g., Bird) requires only modifying the AnimalFactory without changing animal_sound.

No longer directly depends on the Dog or Cat classes. It delegates object creation to the AnimalFactory.

Decoupling Constructors via Factory Design Pattern

```python
# Client Code
def animal_sound(animal_type):
    if animal_type == "dog":
        animal = Dog()  # Tight coupling to the Dog
    elif animal_type == "cat":
        animal = Cat()  # Tight coupling to the Cat
    else:
        raise ValueError("Unknown animal type")

    return animal.speak()
```

```python
# Client Code
def animal_sound(animal_type):
    animal = AnimalFactory.create_animal(animal_type)
    return animal.speak()
```

```python
# Usage
print(animal_sound("dog"))  # Output: Woof!
print(animal_sound("cat"))  # Output: Meow!
```

No longer directly depends on the Dog or Cat classes. It delegates object creation to the AnimalFactory.

Achieving Open-Closed Principle (OCP)

```python
class Bird(Animal):
    def speak(self):
        return "Tweet!"

# Update Factory
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        elif animal_type == "bird":
            return Bird()
        else:
            raise ValueError("Unknown animal type")
```

Open for Extension:

- Add new functionality to the module (e.g., new features or behaviors)

Closed for Modification:

- The existing code of the module should not be modified when adding new functionality

A bit about Static Methods …

```python
class Bird(Animal):
    def speak(self):
        return "Tweet!"

# Update Factory
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        elif animal_type == "bird":
            return Bird()
        else:
            raise ValueError("Unknown animal type")
```

Utility Functions:

- Need a helper function related to the class but *not dependent on instance or class-level data*

Encapsulation:

- To group related logic within the class for better readability and organization

Factory Methods:

- To create objects or perform operations that *don't need access to instance or class-specific data*

Decorator

A bit about Static Methods …

```python
class MathUtils:
    def add(a, b):  # Without self, calling this method will raise an error
        return a + b

# Usage
# MathUtils.add(3, 5)  # Raises TypeError
```

```python
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

# Usage
result = MathUtils.add(3, 5)  # No instance required
print(result)  # Output: 8
```

Utility Functions:

- Need a helper function related to the class but *not dependent on instance or class-level data*

Encapsulation:

- To group related logic within the class for better readability and organization

Factory Methods:

- To create objects or perform operations that *don't need access to instance or class-specific data*

Decorator

Meet its cousins - self and @classmethod

```
class Example:
    class_variable = "Class-Level Data"

    def __init__(self, value):
        self.value = value  # Instance attribute

    # Instance method using self
    def modify_instance_variable(self, new_value):
        self.value = new_value
        print(f"Instance variable modified to: {self.value}")
```

```
    # Class method using cls
    @classmethod
    def modify_class_variable(cls, new_value):
        cls.class_variable = new_value
        print(f"Class variable modified to: {cls.class_variable}")

    # Static method using no self or cls
    @staticmethod
    def utility_method(param):
        print(f"Static method called with param: {param}")
```

```
obj = Example(10)
obj.modify_instance_variable(20)  # Modifies obj's instance variable
```

```
Example.modify_class_variable("Modified by classmethod")
```

```
Example.utility_method("Static call")
```

Animal is inherited by Dog and Cat as “*Base Class*”

```python
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"
```

```python
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Unknown animal type")
```

Why the base class *Animal*?